Processos de Software

DevOps, integração contínua e containers

DIM0510 — Turma 01 · Sprint 2 · aula de 26/10

Prof. Fernando · UFRN · 2026.2

A Sprint 2, a última

	Data
Conteúdo da sprint	a aula de hoje e o vídeo (até 30/10)
Revisão para a prova	04/11 (presencial) e 09/11 (online)
Encontro online de acompanhamento	16/11
Prova escrita	**11/11 (quarta)**, em laboratório
Oficina em sala e *daily* presencial (opcional)	18/11
Daily meetings online	23 e 25/11
Entrega final	**30/11, 23:59**

Critério da Sprint 2	Peso	Aula
Pipeline de CI e containerização	25%	hoje
Métricas DORA	20%	vídeo
Mapeamento de Fluxo de Valor	25%	vídeo
MVP finalizado	15%	—
Retrospectiva final	15%	—

Roteiro de hoje

Min	Bloco	Pergunta
15	DevOps	O que muda quando quem desenvolve também entrega?
25	Integração contínua	Como saber, a cada mudança, que nada quebrou?
10	A proteção da `main`	Como impedir que uma mudança com falha entre na `main`?
15	Containers	Como rodar o projeto do mesmo jeito em qualquer máquina?
25	Oficina	O pipeline e o compose do projeto de vocês

> Hoje fecha um ciclo que começou na Sprint 0: o CI que vocês montaram passa a proteger a `main`: nada entra nela sem passar por ele.

Como acompanhar: as referências

Os exemplos estão no repositório MUSI

Conceito	Onde
Proteção da `main` (e o PR que quebra)	MUSI: `github.com/fmarquesfilho/musi/pull/6`
Workflow de CI	MUSI: `.github/workflows/ci.yml`
`docker compose` e os Dockerfiles	MUSI: `docker-compose.yml` e os Dockerfiles

Abram depois, no navegador, e confiram com calma.

Por que DevOps

Em muitas organizações, quem escreve o código e quem o põe no ar eram equipes diferentes, com objetivos opostos:

	Desenvolvimento	Operação
É cobrado por	entregar mudanças	manter o sistema estável
Vê a mudança como	o trabalho	o risco

- O resultado: entregas raras e grandes, cada uma um evento arriscado
- DevOps propõe tratar entrega e operação como responsabilidade de uma equipe só, com automação e medição
- Não é um cargo nem uma ferramenta: é um conjunto de práticas e de cultura

📖 **Ref.** DORA — Capabilities: continuous integration

O que é (e o que não é) DevOps

Não é	É
"o time de DevOps"	todos fazem entrega e operação
comprar uma ferramenta	mudar o processo e medir
entregar rápido sem qualidade	entregar rápido **e** estável

- **Não é** um cargo, uma equipe nova nem uma ferramenta.
- **É** uma cultura: entregar e operar como responsabilidade **de uma equipe só**, com automação e medição.

📖 **Ref.** Atlassian — CALMS

CALMS: cinco dimensões de DevOps

Letra	Dimensão	O que significa	No projeto de vocês
C	Cultura	responsabilidade compartilhada; falha tratada como aprendizado	a retrospectiva sem culpados
A	Automação	o que se repete vira script ou pipeline	CI, `docker compose`, tasks
L	Lean	fluxo, lotes pequenos, menos desperdício	PRs pequenos, limite de WIP
M	Medição	decidir com dados	métricas de fluxo, DORA
S	Compartilhamento	conhecimento e ferramentas abertos à equipe	README, ADRs, revisão de código

> Quatro das cinco letras já apareceram na disciplina. Hoje o foco é o **A**.

📖 **Ref.** Atlassian — CALMS framework

Os Três Caminhos

Caminho	Ideia	Prática
1. Fluxo	o trabalho anda da esquerda (ideia) para a direita (produção) sem parar	lotes pequenos, integração contínua, limite de WIP
2. Feedback	a informação volta da direita para a esquerda, rápido	testes no CI, monitoramento, revisão
3. Aprendizado contínuo	experimentar e aprender com os erros, todo dia	retrospectivas, tempo para melhorar o processo

- O Primeiro Caminho é o que o Lean e o Kanban já trabalharam: fluxo
- O Segundo é o assunto de hoje: o CI é feedback em minutos, e não na véspera da entrega
- O Terceiro é a retrospectiva: as ações com dono, prazo e verificação

📖 **Ref.** IT Revolution — The Three Ways

Os Três Caminhos em uma frase

1. **Fluxo**: da ideia ao cliente, em lotes pequenos e sem espera.
2. **Feedback**: o erro volta rápido — o CI é feedback em minutos.
3. **Aprendizado**: experimentar e melhorar, todo dia.

> Os três juntos é que são DevOps; um só não basta. Hoje o foco é o Segundo.

Integração contínua

> Cada integrante integra o seu trabalho à linha principal com frequência, pelo menos uma vez por dia, e cada integração é verificada por um build automatizado, com testes.
>
> — Martin Fowler (tradução livre)

Sem CI	Com CI
cada um trabalha dias num ramo próprio	ramos curtos, integrados em horas ou poucos dias
a integração é um evento, perto da entrega	a integração é rotina
"na minha máquina funciona"	funciona numa máquina limpa, a do CI
o problema aparece semanas depois	o problema aparece minutos depois do push

📖 **Ref.** Martin Fowler — Continuous Integration

As práticas da integração contínua

Prática	Na prática, no grupo
todos integram na linha principal com frequência	ramos que vivem horas ou poucos dias, não semanas
o build é automático e verifica a si mesmo	um comando roda compilação e testes; o CI roda o mesmo comando
build quebrado se corrige primeiro	`main` vermelha é a prioridade da equipe, antes de qualquer cartão novo
o build é rápido	feedback em minutos; Fowler sugere manter em torno de 10
todos veem o resultado	o check aparece no PR e na aba Actions

- Ter um servidor de CI não é fazer integração contínua: um ramo que fica três semanas aberto, com CI verde, continua sendo integração tardia
- PRs pequenos são o que torna a integração frequente possível

CI é feedback em minutos

- Integrar na `main` **todo dia** (ou mais), em lotes pequenos.
- O CI devolve o veredito em **minutos**: verde ou vermelho.
- Ramo aberto três semanas, mesmo com CI verde, **não é** integração contínua — é integração tardia.

> O valor do CI é o **loop curto**: erro agora é barato; erro na véspera da entrega, caro.

O ciclo do CI em números

- Alvo do Fowler: um build que roda em **~10 minutos**.
- Ramos que vivem **horas ou poucos dias**, não semanas.
- A cada push, o CI responde **verde ou vermelho** em minutos.

> Se o CI demora 30 minutos, a equipe para de esperar, e o "feedback" deixa de ser rápido.

📖 **Ref.** Martin Fowler — Continuous Integration

Integração, entrega e implantação contínuas

```
commit ⟶ build e testes ⟶ pacote pronto ⟶ ambiente de teste ⟶ produção
```

	Até onde é automático	O que garante	Quem decide ir ao ar
Integração contínua	build e testes	toda mudança na `main` compila e passa nos testes	—
Entrega contínua	até o ambiente de teste	a `main` está sempre pronta para ir ao ar	uma pessoa, com um clique
Implantação contínua	até a produção	toda mudança aprovada vai ao ar	o pipeline

Na disciplina, o mínimo é a integração contínua com a proteção da `main`. Quem implantar (o sistema de Web II, por exemplo) já mede a frequência de implantação no vídeo.

📖 **Ref.** Martin Fowler — Continuous Delivery

O que acontece num pull request

```
push no ramo  ──►  PR aberto  ──►  CI roda  ──►  verde?  ──►  revisão  ──►  merge na main
                                      │
                                      └── vermelho: o merge fica bloqueado até corrigir
```

O que o CI deve rodar, no mínimo:

Etapa	Pergunta que responde
Build	o código compila numa máquina limpa?
Testes	o comportamento continua certo?
Lint	o código segue as regras combinadas?

- O CI roda em **todo** push e PR, não só na `main`
- Rápido importa: um CI de 30 minutos faz a equipe parar de esperar por ele

Anatomia de um workflow do GitHub Actions

```yaml
name: ci                              # o nome que aparece no PR (o status check)

on:                                   # quando roda
  push:
    branches: [main]
  pull_request:

jobs:
  verificar:
    runs-on: ubuntu-latest            # uma máquina limpa, nova a cada execução
    steps:
      - uses: actions/checkout@v5     # baixa o código do commit
      - run: make build               # cada run é um comando de terminal
      - run: make lint
      - run: make test                # se um comando falha, o job fica vermelho
```

- `uses` reaproveita uma ação pronta; `run` executa um comando
- Os comandos são os mesmos que o grupo roda localmente: o CI não deve ter passos que só ele conhece

📖 **Ref.** GitHub Actions — Quickstart

Um CI rápido para os projetos da disciplina

```yaml
jobs:
  kotlin:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-java@v5
        with: { distribution: temurin, java-version: 25, cache: gradle }   # reaproveita as dependências
      - run: ./gradlew build              # compila, roda os testes e o lint configurado
  go:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-go@v6
        with: { go-version-file: servico-go/go.mod }
      - run: go vet ./... && go test ./...
        working-directory: servico-go
```

- Dois jobs rodam em paralelo: o tempo total é o do mais lento
- O cache de dependências tira minutos de cada execução
- Um job por parte do sistema mostra no PR qual parte quebrou

A proteção da `main`

Um CI que só avisa não impede nada. O que impede é a regra de proteção da branch:

Regra	Efeito
Exigir pull request antes do merge	ninguém faz push direto na `main`
Exigir status checks (o job do CI)	o botão de merge fica bloqueado enquanto o CI não está verde
Exigir revisão aprovada (opcional)	outra pessoa precisa aprovar
Incluir administradores	a regra vale para todos, inclusive quem é dono do repositório

No MUSI, hoje, no site: **Settings → Branches**, a regra da `main`.

- o status check `ci` é obrigatório
- o pull request não é exigido
- os administradores não são incluídos (o dono passa por cima)

O CI é obrigatório, mas o PR não, e o dono passa por cima: é a ação A2 da retrospectiva, feita em parte.

📖 **Ref.** GitHub — About protected branches

Demonstração: a proteção funcionando

1. Num ramo, quebrar um teste de propósito e abrir um PR
2. Ver o check do CI ficar vermelho na página do PR
3. Ver o botão de merge bloqueado: *Required status check "ci" has not succeeded*
4. Corrigir, fazer push no mesmo ramo, e ver o check rodar de novo e ficar verde

No site do GitHub: a página do PR mostra o check do CI e o botão de merge bloqueado; a aba **Actions** mostra cada execução do workflow e, dentro dela, o log do passo que falhou.

> A pergunta que a proteção responde: **alguém consegue colocar na `main` um código que quebra os testes?** A resposta certa é não, nem o dono do repositório.

Containers

O problema: o projeto roda numa máquina e não roda em outra, por versão do Java, do banco, de uma biblioteca do sistema.

Termo	O que é
Imagem	um pacote com o programa e tudo de que ele precisa para rodar, inclusive o sistema de arquivos
Container	uma imagem em execução, isolada das outras
`Dockerfile`	a receita da imagem, em texto, versionada com o código
Registro	onde as imagens ficam guardadas para outras máquinas baixarem (Docker Hub, GHCR)

```
FROM eclipse-temurin:25-jre            # a imagem de partida
COPY build/install/app /app            # o que entra na imagem
EXPOSE 8080                            # a porta que o programa usa
CMD ["/app/bin/app"]                   # o comando que roda ao iniciar
```

- `FROM` escolhe a imagem de partida; `COPY` leva arquivos para dentro da imagem; `EXPOSE` apenas **declara** a porta (não a abre); `CMD` é o que roda ao iniciar o container.
- `docker build` executa cada instrução na ordem e guarda o resultado como uma **camada**, empilhada sobre a anterior.
- A imagem é imutável e só de leitura; ao rodar, o container ganha uma camada fina de escrita por cima, que some quando ele para.

📖 **Ref.** Docker — What is a container?

Container não é máquina virtual

	Máquina virtual	Container
o que isola	um computador inteiro, com o próprio sistema operacional	um processo, que usa o núcleo do sistema da máquina
tamanho	gigabytes	de poucos megabytes a centenas
tempo para subir	minutos	segundos
uso típico	rodar outro sistema operacional	empacotar e rodar uma aplicação

- No Mac e no Windows, o Docker roda os containers dentro de uma pequena máquina virtual Linux, e isso é transparente
- O container leva o programa e as bibliotecas; o núcleo é o da máquina que o executa

"Na minha máquina funciona"

O container resolve a frase que mais atrasa projeto:

- A **imagem** empacota o código e as dependências: roda igual em qualquer lugar.
- O `docker compose` sobe tudo (banco, serviço, API) com um comando.
- "Do zero" passa a ser **uma máquina que só tem Docker**.

> Confiram: no MUSI, o `docker-compose.yml` e os Dockerfiles (`api-ktor`, `api-quarkus` e `services`).

Uma imagem em dois estágios

```
# O serviço de busca do MUSI, em Go. Compila na imagem do Go e copia só o
# binário estático para uma imagem mínima (scratch), sem shell.
FROM golang:1.27-alpine AS build
WORKDIR /build
COPY go.mod ./
RUN go mod download               # camada reaproveitada enquanto as dependências não mudam
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -trimpath -ldflags="-s -w" -o /servidor ./cmd/servidor

FROM scratch
COPY --from=build /servidor /servidor
EXPOSE 9090
ENTRYPOINT ["/servidor"]
```

- O compilador fica no primeiro estágio; a imagem final leva só o binário
- Copiar `go.mod` antes do resto aproveita o cache: mudar o código não baixa as dependências de novo
- `scratch`: a imagem final não tem shell nem biblioteca C — só o binário estático

A imagem em camadas

- Cada instrução do `Dockerfile` vira uma **camada**; camadas são reusadas em cache.
- Mudou o código? Só as camadas de cima são reconstruídas; as dependências ficam em cache.
- Por isso `COPY go.mod go.sum` vem **antes** do `COPY . .`: baixar dependências vira uma camada estável.

📖 **Ref.** Docker — multi-stage builds

`docker compose`: o sistema inteiro com um comando

```yaml
services:
  banco:
    image: postgres:17-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U tarefas"]
  api:
    build: .
    ports: ["8080:8080"]
    environment:
      DB_URL: jdbc:postgresql://banco:5432/tarefas   # o nome do serviço é o endereço
    depends_on:
      banco: { condition: service_healthy }          # só sobe com o banco pronto
```

- `services`: cada processo do sistema (banco, API, serviço) é um serviço descrito ali.
- `image` usa uma imagem pronta; `build` constrói a partir de um `Dockerfile`.
- `ports` mapeia a porta do container para a da máquina; `environment` injeta configuração.
- `healthcheck` define "está pronto"; `depends_on: service_healthy` só sobe o dependente quando o banco aceita conexão.

Demonstração — o MUSI: banco, duas APIs (Ktor e Quarkus) e um serviço em Go.

```bash
cd /workspaces/musi
docker compose up --build        # do zero, numa máquina sem Java nem Go instalados
docker compose ps                # banco, api-ktor, api-quarkus e busca: healthy
```

📖 **Ref.** Docker Compose

O que muda na definição de pronto

Antes	Na Sprint 2
funciona na máquina de quem fez	o CI está verde no PR
alguém olhou o código	o PR foi revisado e só integrado com o CI verde
o README explica como instalar	`docker compose up` sobe o projeto do zero

- A rubrica cobra exatamente isso: workflow com build, testes e lint em todo push e PR; verde na `main`; proteção que impede merge com falha; `docker compose up` do zero
- Atualizem o `docs/definicao-de-pronto.md` do grupo

Erros comuns

Erro	Consequência	Como evitar
senha ou token escrito no workflow ou no `compose.yaml`	fica no histórico do Git, visível para quem tiver acesso	*secrets* do repositório; `.env` fora do Git
CI com passos que ninguém roda localmente	"no CI falha, aqui passa"	o CI chama os mesmos comandos do README
teste instável ignorado ("roda de novo que passa")	a equipe deixa de confiar no vermelho	corrigir ou isolar o teste, com cartão no quadro
`depends_on` sem `healthcheck`	a API sobe antes do banco aceitar conexões e cai	`condition: service_healthy`
imagem construída na máquina de alguém	ninguém sabe reproduzir	a imagem sai do `Dockerfile`, no CI ou no `compose`
proteção que exclui administradores	o dono faz merge com o CI vermelho	"incluir administradores" na regra

Sugestão para o grupo

Cada grupo, no próprio repositório, depois da aula:

1. **Workflow.** Conferir que o CI roda build, testes e lint em push e em PR. O que falta?
2. **Proteção.** Em *Settings → Branches* (ou *Rules*), proteger a `main`: PR obrigatório, o job do CI como status check obrigatório, valendo para administradores
3. **Prova.** Abrir um PR que quebra um teste e ver o merge bloqueado
4. **Compose.** Listar o que o `docker compose up` precisa subir no projeto e o que falta

Ao final, respondam: a `main` de vocês aceita hoje um código que quebra os testes?

Da proteção à medição

- A proteção da `main` impede o erro; as **métricas** mostram se o processo melhora.
- O vídeo (até 30/10) cobre DORA e fluxo de valor — e eles precisam de **histórico**.

> Comecem a anotar hoje: cada PR, cada execução do CI, cada implantação. Sem histórico, não há número.

📖 **Ref.** DORA — metrics

Comecem a medir hoje

No vídeo (até 30/10), veremos as métricas DORA e o mapeamento do fluxo de valor. Os dois precisam de **histórico**, e o histórico começa agora:

O que registrar	Onde já fica	Para quê
Cada PR: abertura, revisão, merge	no próprio GitHub	lead time, tempo de espera
Cada execução do CI e o resultado	aba Actions	taxa de falha, tempo de feedback
Cada implantação (deploy)	releases, tags ou o log da plataforma	frequência de implantação
Cada cartão: início e fim	campos de data do quadro (Sprint 1)	tempos de processamento e de espera

> Uma métrica sem método de coleta descrito não conta na rubrica. Anotem desde já **como** cada número foi obtido.

Para conferir o entendimento

1. Um grupo tem CI verde, mas cada integrante trabalha três semanas num ramo próprio. Isso é integração contínua?
2. Qual a diferença entre entrega contínua e implantação contínua?
3. O CI fica vermelho, e o merge acontece assim mesmo. O que falta configurar?
4. Por que a imagem final do serviço Go não tem o compilador?
5. O que `condition: service_healthy` resolve no `docker compose`?

Onde estudar depois

- Leitura da sprint: `leituras/processos-s2.md`, capítulos 1 a 4
- Martin Fowler — Continuous Integration
- DORA — Continuous integration
- IT Revolution — The Three Ways
- GitHub — About protected branches
- Docker — What is a container? · Docker Compose